

EE/CS 148B HW 2

Profiling and Reasoning*

Lead TAs: Damiano Marsili, Vansh Tibrewal

Spring 2026

1 Assignment Overview

In this assignment, you will gain hands-on experience at profiling and optimizing model speeds, as well as experiment with prompting and post-training for math reasoning.

What you will implement.

1. Benchmarking and profiling harness (§2)
2. Evaluation harness (§3.1)
3. Zero-shot prompting baselines on the GSM8K dataset (§3.2)
4. GRPO post-training on the GSM8K dataset (§3.5)

What the code looks like. All the assignment code is hosted on GitHub at <https://github.com/caltech-eecs148b/hw2>. For details on the repository layout, please refer to the `README.md` file in the repository.

Where to get the data. This assignment will use the GSM8K dataset. We recommend using Huggingface's `datasets` to access it at the following link: <https://huggingface.co/datasets/openai/gsm8k>

How to submit. You will submit the following files to Gradescope:

- `writeup.pdf`: Answer all the written questions. Please typeset your responses.
- `code.zip`: Contains all the code you've written. Create a **private** repository on GitHub and push your code to it. Submit this GitHub repo by following the directions on Gradescope.

*This assignment draws from HW2 and HW5 of Stanford's CS336 course by Percy Liang and Tatsunori Hashimoto: <https://github.com/stanford-cs336>. We adapt the assignment for this course.

Note on Compute. We assume students have access to Colab Pro+, which provides access to A100 and H100 GPUs. You can swap the GPU in Colab via "Runtime" → "Change Runtime Type". This plan comes with "compute units" and allocations, so we recommend the following usage:

- For benchmarking (§2), profiling (§2), and zero-shot baselines (§3.2) we recommend using the "free" tier GPUs that should frequently be available (T4, L4, or G4). These analyses are not compute intensive and a correct implementation should work on these machines.
- For post-training (§3.5) we recommend using an A100 or H100 GPU. We have tested the implementation on both and all experiments can be run on either GPU in around one hour. Accessing an A100 should be more reliable than an H100, as H100s are more limited.

We will be reimbursing students for subscriptions to Google Colab Pro+ at the end of the quarter. Please keep your receipts for the reimbursement process. Alternatively, if you have other means of compute that you would like to use, feel free to.

2 Profiling and Benchmarking

In the first part of the assignment, we will look into how to optimize the performance of our Transformer model to make the most efficient use of the GPU. We will profile our model to understand where it spends time and memory during the forward and backward passes.

Before implementing any optimization, it is helpful to first profile our program to understand where it spends resources (e.g., time and memory). Otherwise, we risk optimizing parts of the model that don't account for significant time or memory, and therefore not seeing measurable end-to-end improvements. We will implement three performance evaluation paths: (a) a simple, end-to-end benchmarking using the Python standard library to time our forward and backward passes, (b) profile compute with the NVIDIA Nsight Systems tool to understand how that time is distributed across operations on both the CPU and GPU, and (c) profile memory usage.

2.1 Setup - Importing the Basics Transformer Model

Let's start by making sure that you can load the model from the previous assignment. In the previous assignment, we set up our model in a Python package, so that it could be easily imported later. We have added parts of the staff implementation of the model in the `basics` folder, and have pointed to it in the `pyproject.toml` file. By calling `uv run [command]` as usual, `uv` will automatically locate this local `basics` package. You can test that you can import your model with:

```
~$ uv run python
Using CPython 3.12.10
Creating virtual environment at: /path/to/uv/env/dir
...
Installed 85 packages in 711ms
Python 3.12.10 (main, Apr 9 2025, 04:03:51) [Clang 20.1.0 ] on linux
...
>>> import basics
>>>
```

2.2 Model Sizing

Throughout this assignment, we will be benchmarking and profiling models to better understand their performance. To get a sense of how things change at scale, we will work with and refer to the following model configurations. For all models, we'll use a vocabulary size of 10,000 and a batch size of 4. Unless specified, use a context length of 128.

Size	d_model	d_ff	num_layers	num_heads
small	512	2048	8	8
medium	768	3072	12	12
large	1024	4096	24	16

Table 1: Specifications of different model sizes

2.3 End-to-End Benchmarking

We will now implement a simple performance evaluation script. We will be testing many variations of our model (changing precision, swapping layers, etc.), so it will pay off to have your script enable these variations via command-line arguments to make them easy to run later on. To start off, let's do the simplest possible profiling of our model by timing the forward and backward passes. Since we will only be measuring speed and memory, we will use random weights and data. Unless specified, you should use a context length of 128.

Measuring performance is subtle — some common traps can cause us to not measure what we want. For benchmarking GPU code, one caveat is that CUDA calls are asynchronous. When you call a CUDA kernel, such as when you invoke `torch.matmul`, the function call returns control to your code without waiting for the matrix multiplication to finish. In this way, the CPU can continue running while the GPU computes the matrix multiplication. On the other hand, this means that naïvely measuring how long the `torch.matmul` call takes to return does not tell us how long the GPU takes to actually run the matrix multiplication. In PyTorch, we can call `torch.cuda.synchronize()` to wait for all GPU kernels to complete, allowing us to get more accurate measurements of CUDA kernel runtime. With this in mind, let's write our basic profiling infrastructure.

Problem (benchmarking_script): Simple profiling

(1): Write a script to perform basic end-to-end benchmarking of the forward and backward passes in your model. Specifically, your script should support the following:

- Given hyperparameters (e.g., number of layers), initialize a model.
- Generate a random batch of data.
- Run w warm-up steps (before you start measuring time), then time the execution of n steps (either only forward, or both forward and backward passes, depending on an argument). For timing, you can use the Python `timeit` module (e.g., either using the `timeit` function, or using `timeit.default_timer()`, which gives you the system's highest resolution clock, thus a better default for benchmarking than `time.time()`).
- Call `torch.cuda.synchronize()` after each step.

Deliverable: A script that will initialize a basics Transformer model with the given

hyperparameters, create a random batch of data, and time forward and backward passes.

- (2): Time the forward and backward passes for the model sizes described in Table 1. Use 5 warmup steps and compute the average and standard deviation of timings over 10 measurement steps. How long does a forward pass take? How about a backward pass? Which of the two takes longer, and why? Do you see high variability across model sizes, or is the standard deviation small?

Deliverable: A 1-2 sentence response with your timings.

- (3): One caveat of benchmarking is not performing the warm-up steps. Repeat your analysis without the warm-up steps. How does this affect your results? Why do you think this happens? Also try to run the script with 1 or 2 warm-up steps. Why might the result still be different? What about 10 warm-up steps?

Deliverable: A 2-3 sentence response.

2.4 Nsight Systems Profiler

End-to-end benchmarking does not tell us where our model spends time and memory during forward and backward passes, and so does not expose specific optimization opportunities. To know how much time our program spends in each component (e.g., function), we can use a profiler. An execution profiler instruments the code by inserting guards when functions begin and finish running, and thus can give detailed execution statistics at the function level (such as number of calls, how long they take on average, cumulative time spent on this function, etc).

Standard Python profilers (e.g., `CProfile`) are not able to profile CUDA kernels since these kernels are executed asynchronously on the GPU. Fortunately, NVIDIA ships a profiler that we can use via the CLI `nsys`, which we have already installed for you. In this part of the assignment, you will use `nsys` to analyze the runtime of your Transformer model. Using `nsys` is straightforward: we can simply run your Python script from the previous section with `nsys` profile prepended. For example, you can profile a script `benchmark.py` and write the output to a file `result.nsys.rep` with:

```
uv run nsys profile -o result python benchmark.py
```

You can then view the profile on your local machine with the NVIDIA Nsight Systems desktop application. Selecting a particular CUDA API call (on the CPU) in the CUDA API row of the profile will highlight all corresponding kernel executions (on the GPU) in the CUDA HW row.

We encourage you to experiment with various command-line options for `nsys` profile to get a sense of what it can do. Notably, you can get Python backtraces for each CUDA API call with `--python-backtrace=cuda`, though this may introduce overhead. You can also annotate your code with NVTX ranges, which will appear as blocks in the NVTX row of the profile capturing all CUDA API calls and associated kernel executions. In particular, you should use NVTX ranges to ignore the warm-up steps in your benchmarking script (by applying a filter on the NVTX row in the profile). You can also isolate which kernels are responsible for the forward and backward passes of your model, and you can even isolate which kernels are responsible for different parts of a self-attention layer by annotating your implementation as follows:

```
...  
import torch.cuda.nvtx as nvtx
```

```

@nvtx.range("scaled dot product attention")
def annotated_scaled_dot_product_attention(
    ... # Q, K, V, mask
)
    ...

    with nvtx.range("computing attention scores"):
        ... # compute attention scores between Q and K

    with nvtx.range("computing softmax"):
        ... # compute softmax of attention scores

    with nvtx.range("final matmul"):
        ... # compute output projection

    return ...

```

You can swap your original implementation with the annotated version in your benchmarking script via:

```

basics.model.scaled_dot_product_attention = annotated_scaled_dot_product_attention

```

Finally, you can use the `--pytorch` command-line option with `nsys` to automatically annotate calls to the PyTorch C++ API with NVTX ranges.

Problem (nsys_profile): NVTX profiling

- (1): Profile your forward pass, backward pass, and optimizer step using `nsys` with each of the model sizes described in Table 1 and context lengths of 32, 64, 128 and 256 (you may run out of memory with some of these context lengths, in which case just note it in your report).
 1. What is the total time spent on your forward pass? Does it match what we had measured before with the Python standard library?
 2. What CUDA kernel takes the most cumulative GPU time during the forward pass? How many times is this kernel invoked during a single forward pass of your model? Is it the same kernel that takes the most runtime when you do both forward and backward passes? (**Hint:** look at the “CUDA GPU Kernel Summary” under “Stats Systems View”, and filter using NVTX ranges to identify which parts of the model are responsible for which kernels.)
 3. Although the vast majority of FLOPs take place in matrix multiplications, you will notice that several other kernels still take a non-trivial amount of the overall runtime. What other kernels besides matrix multiplies do you see accounting for non-trivial CUDA runtime in the forward pass?
 4. Profile running one complete training step with AdamW (i.e., the forward pass, computing the loss and running a backward pass, and finally an optimizer step, as you’d do during training). How does the fraction of time spent on matrix multiplication change, compared to doing inference (forward pass only)? How about other kernels?

5. Compare the runtime of the softmax operation versus the matrix multiplication operations within the self-attention layer of your model during a forward pass. How does the difference in runtimes compare to the difference in FLOPs?

Deliverable: 1-2 sentence responses for each of the subquestions.

2.5 Mixed Precision

Up to this point in the assignment, we've been running with FP32 precision—all model parameters and activations have the `torch.float32` datatype. However, modern NVIDIA GPUs contain specialized GPU cores (Tensor Cores) for accelerating matrix multiplies at lower precisions. For example, the NVIDIA A100 spec sheet says that its maximum throughput with FP32 is 19.5 TFLOP/second, while its maximum throughput with FP16 (half-precision floats) or BF16 (brain floats) is significantly higher at 312 TFLOP/second. As a result, using lower-precision datatypes should help us speed up training and inference.

However, naïvely casting our model into a lower-precision format may come with reduced model accuracy. For example, many gradient values in practice are often too small to be representable in FP16, and thus become zero when naïvely training with FP16 precision. To combat this, it's common to use loss scaling when training with FP16—the loss is simply multiplied by a scaling factor, increasing gradient magnitudes so they don't flush to zero. Furthermore, FP16 has a lower dynamic range than FP32, which can lead to overflows that manifest as a NaN loss. Full bfloat16 training is generally more stable (since BF16 has the same dynamic range as FP32), but can still affect final model performance compared to FP32.

To take advantage of the speedups from lower-precision datatypes, it's common to use mixed-precision training. In PyTorch, this is implemented with the `torch.autocast` context manager. In this case, certain operations (e.g., matrix multiplies) are performed in lower-precision datatypes, while other operations that require the full dynamic range of FP32 (e.g., accumulations and reductions) are kept as-is. For example, the following code will automatically identify which operations to perform in lower-precision during the forward pass and cast these operations to the specified data type:

```
model : torch.nn.Module = ... # e.g. your Transformer model
dtype : torch.dtype = ... # e.g. torch.float16
x : torch.Tensor = ... # input data

with torch.autocast(device="cuda", dtype=dtype):
    y = model(x)
```

As alluded to above, it is generally a good idea to keep accumulations in higher precision even if the tensors themselves being accumulated have been downcasted. The following exercise will help build your intuition as to why this is the case.

Problem (mixed_precision_accumulation): Effect of mixed precision

Run the following code and comment on the (accuracy of the) results:

```
s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01, dtype=torch.float32)
print(s)
```

```

s = torch.tensor(0, dtype=torch.float16)
for i in range(1000):
    s += torch.tensor(0.01, dtype=torch.float16)
print(s)

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01, dtype=torch.float16)
print(s)

s = torch.tensor(0, dtype=torch.float32)
for i in range(1000):
    x = torch.tensor(0.01, dtype=torch.float16)
    s += x.type(torch.float32)
print(s)

```

Deliverable: A 2-3 sentence response.

We now apply mixed precision to our benchmarking harness to evaluate it's impact on performance.

Problem (mixed_precision_benchmarking): Benchmarking mixed precision.

(1): Consider the following Toy Model:

```

class ToyModel(nn.Module):
    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 10, bias=False)
        self.ln = nn.LayerNorm(10)
        self.fc2 = nn.Linear(10, out_features, bias=False)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.ln(x)
        x = self.fc2(x)
        return x

```

Suppose we are training the model on a GPU and that the model parameters are originally in FP32. We'd like to use autocasting mixed precision with FP16. What are the data types of:

- the model parameters within the autocast context,
- the output of the first feed-forward layer (ToyModel.fc1),
- the output of layer norm (ToyModel.ln),
- the model's predicted logits,
- the loss
- the model gradients

Deliverable: The datatype of each listed component.

- (2): You should have seen that FP16 mixed precision autocasting treats the layer normalization layer differently than the feed-forward layers. What parts of layer normalization are sensitive to mixed precision? If we use BF16 instead of FP16, do we still need to treat layer normalization differently? Why or why not?

Deliverable: A 2-3 sentence response.

- (3): Modify your benchmarking script to optionally run the model using mixed precision with BF16. Time the forward and backward passes with and without mixed-precision for each language model size described in Table 1. Compare the results of using full vs. mixed precision, and comment on any trends as model size changes. You may find the `nullcontext` no-op context manager to be useful.

Deliverable: A 2-3 sentence response with your timings and commentary.

2.6 Memory Profiling

So far, we have been looking at compute performance. We'll now shift our attention to memory, another major resource in language model training and inference. PyTorch also ships with a powerful memory profiler, which can keep track of allocations over time.

To use the memory profiler, you can modify your benchmarking script as follows:

```
... # warm-up phase in your benchmarking script

# Start recording memory history.
torch.cuda.memory._record_memory_history(max_entries=1000000)

... # what you want to profile in your benchmarking script

# Save a pickle file to be loaded by PyTorch's online tool.
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")

# Stop recording history.
torch.cuda.memory._record_memory_history(enabled=None)
```

This will output a file `memory_snapshot.pickle` that you can load into the following online tool: https://pytorch.org/memory_viz. This tool will let you see the overall memory usage timeline as well as each individual allocation that was made, with its size and a stack trace leading to the code where it originates. To use this tool, you should open the link above in a Web browser, and then drag and drop your Pickle file onto the page.

You will now use the PyTorch profiler to analyze the memory usage of your model.

Problem (memory_profiling): Profiling memory

Profile your forward pass, backward pass, and optimizer step of the **large** model from Table 1.

- (1) Add an option to your profiling script to run your model through the memory profiler. It may be helpful to reuse some of your previous infrastructure (e.g., to activate mixed-precision, load specific model sizes, etc). Then, run your script to get a memory profile

of the **large** model when either doing inference only (just forward pass) or a full training step. How do your memory timelines look like? Can you tell which stage is running based on the peaks you see?

Deliverable: Two images of the “Active memory timeline” of a **large** model, from the `memory_viz` tool: one for the forward pass, and one for running a full training step (forward and backward passes, then optimizer step), and a 2-3 sentence response.

- (2) What is the peak memory usage of each context length when doing a forward pass? What about when doing a full training step?

Deliverable: A table with two numbers per context length.

- (3) Find the peak memory usage of the **large** model when using mixed-precision, for both a forward pass and a full optimizer step. Does mixed-precision significantly affect memory usage?

Deliverable: A 2-3 sentence response.

- (4) Consider the **large** model. At our reference hyperparameters, what is the size of a tensor of activations in the Transformer residual stream, in single-precision? Give this size in MB (i.e., divide the number of bytes by 1024^2).

Deliverable: A 1-2 sentence response with your derivation.

2.7 Profiling Attention

Your profiling likely suggests that there is an opportunity for optimization, both in terms of memory and compute, in your attention layers. At a high level, the attention operation consists of a matrix multiplication followed by softmax, then another matrix multiplication:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\text{mask}\left(\frac{Q^\top K}{\sqrt{d_k}}\right)\right) V \quad (1)$$

The naïve attention implementation needs to save attention score matrices of shape `seq_len × seq_len` for each batch/head element, which can grow very large with long sequence lengths, causing out-of-memory errors for any tasks with long inputs or outputs.

Problem (attention_profiling): Attention profiling

- (1) Benchmark your attention implementation at different scales. Write a script that will:

- Fix the batch size to 8 and don’t use multihead attention (i.e. set `num_heads` to 1).
- Iterate through the cartesian product of `[16, 32, 64, 128]` for the head embedding dimension `d_model`, and `[64, 128, 256, 512, 1024]` for the sequence length.
- Create random inputs Q, K, V for the appropriate size.
- Time 100 forward passes through attention using the inputs.
- Measure how much memory is in use before the backward pass starts, and time 100 backward passes.

- Make sure to warm up, and to call `torch.cuda.synchronize()` after each forward/backward pass.

Report the timings (or out-of-memory errors) you get for these configurations. At what size do you get out-of-memory errors? Do the accounting for the memory usage of attention in one of the smallest configurations you find that runs out of memory (you can use the equations for memory usage of Transformers from Assignment 1). How does the memory saved for backward change with the sequence length? What would you do to eliminate this memory cost?

Deliverable: A table with your timings, your working out for the memory usage, and a 1-2 paragraph response.

2.8 Benchmarking JIT-Compiled Attention

Since version 2.0, PyTorch also ships with a powerful just-in-time compiler that automatically tries to apply a number of optimizations to PyTorch functions: see https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html for an intro. In particular, it will try to automatically generate fused Triton kernels by dynamically analyzing your computation graph. The interface to use the PyTorch compiler is very simple. For instance, if we wanted to apply it to a single layer of our model, we can use:

```
layer = SomePyTorchModule(...)
compiled_layer = torch.compile(layer)
```

Now, `compiled_layer` functionally behaves just like `layer` (e.g., with its forward and backward passes). We can also compile our entire PyTorch model with `torch.compile(model)`, or even a Python function that calls PyTorch operations.

Problem (torch.compile): Benchmarking JIT-Compiled Attention

- (1) Extend your attention benchmarking script to include a compiled version of your PyTorch implementation of attention, and compare its performance to the uncompiled version with the same configuration as the `attention_profiling` problem above.

Deliverable: A table comparing your forward and backward pass timings for your compiled attention module with the uncompiled version from the `attention_profiling` problem above.

- (2) Now, compile your entire Transformer model in your end-to-end benchmarking script. How does the performance of the forward pass change? What about the combined forward and backward passes and optimizer steps?

Deliverable: A table comparing your vanilla and compiled Transformer model.

As you have observed, naive attention is computationally expensive, particularly at large sequence lengths. A recent method to improve the computational efficiency of Attention is called Flash-Attention. While Flash Attention is out of the scope of this assignment, we recommend students that are interested to explore [Stanford's CS336 assignment 2](#) (Section 1.3).

3 Reasoning and RL

We now turn to improving our LLM with post-training. You will gain hands-on experience with setting up prompting baselines and training LLMs with GRPO.

What you will implement.

1. Direct prediction baseline for the GSM8K dataset of math problems by Cobbe et. Al [1].
2. Zero-shot baselines: Chain-of-Thought [2] and Self-Consistency [3].
3. Group Relative Policy Optimization (GRPO; [4]) for improving reasoning performance with verified rewards.

What you can use. We expect you to build most of the RL related components from scratch. You may use tools like vLLM to generate text from language models (§ 3.1.1). In addition, you may use HuggingFace Transformers to load the Qwen 2.5 Math 1.5B model and tokenizer and run forward passes, but you may not use any of the training utilities (e.g., the Trainer class).

There are going to be two differences from the way we’ve done our past assignments.

1. First, we are not going to be using our language model codebase and models from earlier. We would ideally like to use base language models trained from previous assignments, but finetuning those models will not give us a satisfying result—these models are far too weak to display non-trivial mathematical reasoning capabilities. Because of this, we are going to switch to a modern, high-performance language model that we can access (Qwen 2.5 Math 1.5B Base) and do most of our work on top of that model.
2. Second, we are going to introduce a new benchmark with which to evaluate our language models. Up until this point, we have embraced the view that cross-entropy is a good surrogate for many downstream tasks. However, the point of this assignment will be to bridge the gap between base models and downstream tasks and so we will have to use evaluations that are separate from cross-entropy. We will use the GSM8K dataset from Cobbe et al. [1], which consists of challenging high-school competition mathematics problems. We will evaluate language model outputs by comparing them against a reference answer.

3.1 Measuring Direct Prediction Performance

We’ll start by measuring the performance of our base language model on the 1.32K examples of GSM8K test set.

To begin, we will measure direct prediction performance without any sophisticated prompting. Specifically, we will use the following prompt:

```
prompt = "Please answer with ONLY the answer enclosed in <answer> </answer> tags. \n
         Question: {question}"
```

In the prompt, we will replace `question` with the question we are asking the LLM. The purpose of having the model generate tags like `<answer> </answer>` is so that we can easily parse the model’s output and compare it against a ground truth answer, and so that we can stop response generation when we see the right answer tag `</answer>`.

3.1.1 Using vLLM for offline language model inference.

To evaluate our language models, we're going to have to generate continuations (responses) for a variety of prompts. While one could certainly implement their own functions for generation (e.g., as you did in assignment 1), efficient implementation of RL requires high-performance inference techniques, and implementing these inference techniques are beyond the scope of this assignment. Therefore, in this assignment we will recommend using vLLM for offline batched inference. vLLM is a high-throughput and memory-efficient inference engine for language models that incorporates a variety of useful efficiency techniques (e.g., optimized CUDA kernels, PagedAttention for efficient attention KV caching). To use vLLM to generate continuations for a list of prompts:

```
from vllm import LLM, SamplingParams

# Sample prompts.
prompts = [
    "Hello, my name is",
    "The president of the United States is",
    "The capital of France is",
    "The future of AI is",
]

# Create a sampling params object, stopping generation on newline.
sampling_params = SamplingParams(
    temperature=1.0, top_p=1.0, max_tokens=1024, stop=["\n"]
)

# Create an LLM.
llm = LLM(model=<path to model>)

# Generate texts from the prompts. The output is a list of RequestOutput objects
# that contain the prompt, generated text, and other information.
outputs = llm.generate(prompts, sampling_params)

# Print the outputs.
for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

In the example above, the LLM can be initialized with the name of a HuggingFace model (which will be automatically downloaded and cached if it isn't found locally), or a path to a HuggingFace model. For the rest of this assignment, you should load the Qwen 2.5 Math 1.5B Base at <https://huggingface.co/Qwen/Qwen2.5-Math-1.5B>.

Note: for this assignment, using vLLM is **optional**. You may implement the baselines in Section 3.1 using the `transformers` package directly. However, we provide the above code for students that choose to experiment with vLLM. If you do, it may be interesting to explore the relative speedup provided by vLLM.

3.1.2 Direct prediction baseline.

Prompting. To evaluate zero-shot performance on the GSM8K test set, we'll simply load the examples and prompt the language model to answer the question using the prompt from above.

Evaluation metric. When we evaluate a multiple-choice or binary response task, the evaluation metric is clear—we test whether the model outputs exactly the correct answer. In math problems we assume that there is a known ground truth (e.g. 0.5) but we cannot simply test whether the model outputs exactly 0.5—it can also answer `<answer> 1/2 </answer>`. Because of this, we must address the tricky problem of matching for semantically equivalent responses from the LM when we evaluate GSM8K.

To this end, we want to come up with some answer parsing function that takes as input the model’s output and a known ground-truth, and returns a boolean indicating whether the model’s output is correct. For example, a reward function could receive the model’s string output ending in `<answer> She sold 15 clips.</answer>` and the gold answer 72, and return True if the model’s output is correct and False otherwise (in this case, it should return False).

For our GSM8K experiments, we will use a fast and fairly accurate answer parser used in recent work on reasoning RL [11]. This reward function is implemented at `alignment/drgrepo_grader.py`, named `r1_zero_reward_fn` and you should use it to evaluate performance on GSM8K unless otherwise specified.

Generation hyperparameters. When generating responses, we’ll sample with temperature 1.0, top-p 1.0, max generation length 1024. The prompt asks the model to end its answer with the string `</answer>`, and therefore we can direct vLLM to stop when the model outputs this string:

```
# Based on Dr. GRP0: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
# c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params.stop = ["</answer>"]
sampling_params.include_stop_str_in_output = True
```

Problem (gsm_baseline): Direct prompting

- (1): Write a script to evaluate Qwen 2.5 Math 1.5B zero-shot performance on GSM8K. This script should (1) load the GSM8K dataset from <https://huggingface.co/datasets/openai/gsm8k> (make sure to load the `test` split), (2) format them as string prompts to the language model using the given prompt, and (3) generate outputs for each example. This script should also (4) calculate evaluation metrics and (5) serialize the examples, model generations, and corresponding evaluation scores to disk for analysis in subsequent problems. It might be helpful for your implementation to include a method `evaluate_vllm` with arguments similar to the following, as you will be able to reuse it later:

```
def evaluate_vllm(
    vllm_model: LLM,
    reward_fn: Callable[[str, str], dict[str, float]],
    prompts: List[str],
    eval_sampling_params: SamplingParams
) -> None:
    """
    Evaluate a language model on a list of prompts,
    compute evaluation metrics, and serialize results to disk.
    """
```

Deliverable: A script to evaluate baseline direct prediction GSM8K performance.

Note: For students who choose to use `transformers` instead of `vLLM`, we recommend implementing a similar function. As always, the details of your implementation/structure are up to you.

- (2): Run your evaluation script on Qwen 2.5 Math 1.5B. How many model generations fall into each of the following categories: (1) correct with both format and answer reward 1, (2) format reward 1 and answer reward 0, (3) format reward 0 and answer reward 0? Observing at least 10 cases where format reward is 0, do you think the issue is with the base model’s output, or the parser? Why? What about in (at least 10) cases where format reward is 1 but answer reward is 0?

Deliverable: Commentary on the model and reward function performance, including examples of each category.

- (3): How well does the Qwen 2.5 Math 1.5B zero-shot baseline perform on GSM8K?

Deliverable: 1-2 sentences with evaluation metrics

3.2 Zero-shot Prompting Baselines

Before we turn to post-training our Qwen 2.5 model, we explore additional zero-shot prompting techniques to explore “how far” we can push the base model. We focus on two approaches: Chain-of-Thought and Self-Consistency, which we describe next.

Chain-of-Thought (CoT). Early work on LLM reasoning demonstrated that LLMs perform better on reasoning tasks when decomposing the problem into steps and using a text-based “scratch-pad” [2]. We can elicit this behavior by prompting the model to “think step-by-step”. To experiment with CoT, we replace our earlier prompt with the following one:

```
'''A conversation between User and Assistant. The User asks a question, and the
Assistant solves it. The Assistant first thinks about the reasoning process in the
mind and then provides the User with the answer. The reasoning process is enclosed
within <think> </think> and answer is enclosed within <answer> </answer> tags,
respectively, i.e., <think> reasoning process here </think> <answer> answer here </
answer>.
User: {question}
Assistant: <think>
'''
```

This prompt can be found inside `alignment/prompts.py`.

Self-Consistency. Given LLM sampling is stochastic, a competent LLM may still sample the incorrect answer due to change. Thus, with self-consistency, we operate on the premise that if an LLM knows the correct answer it should be sampled *more frequently* than an incorrect one. Specifically, we sample K responses for each query and take a majority vote for our final prediction.

Problem (prompting_baselines): CoT and Self-Consistency

- (1): Evaluate the Qwen 2.5 Math 1.5B model on GSM8K with chain-of-thought prompting. How does performance differ from direct prompting? Examine some model predictions,

how faithful are the reasoning traces to the final responses? Is the model always internally consistent?

- (2): Evaluate the Qwen 2.5 Math 1.5B model on GSM8K with self-consistency. Use $K = 5$ and the **CoT prompt**. How does performance compare to single-shot direct prompting? Examine some model predictions, how often are there ties? How uni-modal are the model predictions?

Deliverable: 2-3 sentence responses for each of the questions.

3.3 Post-Training Utilities

We now turn to improving the reasoning capabilities of the model with reinforcement learning. Before setting up our reinforcement learning loop, we setup some utilities we will need.

3.3.1 Using Huggingface Models

Loading a HuggingFace model and tokenizer. To load a HuggingFace model and tokenizer (in bfloat16) you can use the following starter code:

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-Math-1.5B",
    torch_dtype=torch.bfloat16,
)
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-Math-1.5B")
```

Optionally, you may explore loading the model with with FlashAttention-2 to save memory, but this requires installing flash-attn.

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-Math-1.5B",
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",
)
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-Math-1.5B")
```

Forward pass. After we've loaded the model, we can run a forward pass on a batch of input IDs and get the logits (with the `.logits`) attribute of the output. Then, we can compute a loss:

```
input_ids = train_batch["input_ids"].to(device)

logits = model(input_ids).logits
loss = ...
```

Saving a trained model. To save the model to a directory after training is finished, you can use the `.save_pretrained()` function, passing in the path to the desired output directory. We recommend also saving the tokenizer as well (even if you didn't modify it), just so the model and tokenizer are self-contained and loadable from a single directory.

```
# Save the model weights
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

Gradient accumulation. Despite loading the model in `bfloat16`, even an 80GB GPU does not have enough memory to support reasonable batch sizes. To use larger batch sizes, we can use a technique called **gradient accumulation**. The basic idea behind gradient accumulation is that rather than updating our model weights (i.e., taking an optimizer step) after every batch, we'll accumulate the gradients over several batches before taking a gradient step. Intuitively, if we had a larger GPU, we should get the same results from computing the gradient on a batch of 32 examples all at once, vs. splitting them up into 16 batches of 2 examples each and then averaging at the end.

Gradient accumulation is straightforward to implement in PyTorch. Recall that each weight tensor has an attribute `.grad` that stores its gradient. Before we call `loss.backward()`, the `.grad` attribute is `None`. After we call `loss.backward()`, the `.grad` attribute contains the gradient. Normally, we'd take an optimizer step, and then zero the gradients with `optimizer.zero_grad()`, which resets the `.grad` field of the weight tensors:

```
for inputs, labels in data_loader:
    # Forward pass.
    logits = model(inputs)
    loss = loss_fn(logits, labels)

    # Backward pass.
    loss.backward()

    # Update weights.
    optimizer.step()
    # Zero gradients in preparation for next iteration.
    optimizer.zero_grad()
```

To implement gradient accumulation, call the `optimizer.step()` and `optimizer.zero_grad()` every k steps, where k is the number of gradient accumulation steps. We divide the loss by `gradient_accumulation_steps` before calling `loss.backward()` so that the gradients are averaged across the gradient accumulation steps.

```
gradient_accumulation_steps = 4
for idx, (inputs, labels) in enumerate(data_loader):
    # Forward pass.
    logits = model(inputs)
    loss = loss_fn(logits, labels) / gradient_accumulation_steps

    # Backward pass.
    loss.backward()

    if (idx + 1) % gradient_accumulation_steps == 0:
        # Update weights every 'gradient_accumulation_steps' batches.
        optimizer.step()

        # Zero gradients every 'gradient_accumulation_steps' batches.
        optimizer.zero_grad()
```

As a result, our effective batch size when training is multiplied by k , the number of gradient accumulation steps.

3.3.2 Helper Methods

Next, we will implement some helper methods that you will use during our RL experiments. As a quick note on nomenclature: in the following sections, we will interchangeably refer to a model's completion given a prompt as an “output”, “completion”, or “response”.

Tokenizing prompts and outputs. For each pair of question and target output (q, o) , we will tokenize the question and output separately and concatenate them. Then, we can score the log-probabilities of the output with our RL policy. Moreover, we will need to construct a `response_mask`: a boolean mask that is True for all tokens in the response, and False for all question and padding tokens. We will use this mask in the training loop to ensure that we only compute the loss on the response tokens.

Problem (`tokenize_prompt_and_output`): Tokenization.

Deliverable: Implement a method `tokenize_prompt_and_output` that tokenizes the question and output separately, concatenates them together, and constructs a `response_mask`. The following interface is recommended:

```
def tokenize_prompt_and_output(prompt_strs, output_strs, tokenizer):
    """
    Tokenize the prompt and output strings, and construct a mask that is 1
    for
    the response tokens and 0 for other tokens (prompt or padding).

    Args:

        prompt_strs: list[str] List of prompt strings.

        output_strs: list[str] List of output strings.

        tokenizer: PreTrainedTokenizer Tokenizer to use for tokenization.

    Returns:

        dict[str, torch.Tensor]. Let prompt_and_output_lens be a list
        containing
        the lengths of the tokenized prompt and output strings. Then the
        returned
        dictionary should have the following keys:

            input_ids torch.Tensor of shape (batch_size, max(
                prompt_and_output_lens) - 1):
                the tokenized prompt and output strings, with the final token
                sliced off.

            labels torch.Tensor of shape (batch_size, max(
                prompt_and_output_lens) - 1):
                shifted input ids, i.e., the input ids without the first
                token.

            response_mask torch.Tensor of shape (batch_size, max(
```

```

prompt_and_output_lens) - 1):
    a mask on the response tokens in the labels.
"""

```

To test your code, implement `[adapters.run_tokenize_prompt_and_output]`. Then, run the test with `uv run pytest -k test_tokenize_prompt_and_output` and make sure your implementation passes it.

Logging per-token entropies. When doing RL, it is often useful to keep track of per-token entropies to see if the predictive distribution of the model is becoming (over)confident. We will implement this now and compare how each of our finetuning approaches affects the model’s predictive entropy. The entropy of a discrete distribution $p(x)$ with support $\mathcal{X}$ is defined as:

$$H(p) = - \sum_{x \in \mathcal{X}} p(x) \log p(x) \quad (2)$$

Given our RL model’s logits, we will compute the per-token entropy, i.e., the entropy of each next-token prediction.

Problem (compute_entropy): Computing model entropies.

Deliverable: Implement a method `compute_entropy` that computes the per-token entropy of next-token predictions. The following interface is recommended:

```

def compute_entropy(logits: torch.Tensor) -> torch.Tensor:
    """
    Get the entropy of the next-token predictions (i.e., entropy over the
    vocabulary dimension).

    Args:
        logits: torch.Tensor Tensor of shape (batch_size, sequence_length,
            vocab_size)
            containing unnormalized logits.

    Returns:
        torch.Tensor Shape (batch_size, sequence_length). The entropy for
        each next-token
        prediction.
    """

```

Note: you should use a numerically stable method (e.g., using `logsumexp`) to avoid overflow. To test your code, implement `[adapters.run_compute_entropy]`. Then run `uv run pytest -k test_compute_entropy` and ensure your implementation passes.

Getting log-probabilities from a model. Obtaining log-probabilities from a model is a primitive that we will need for RL.

For a prefix x , an LM producing next-token logits $f_{\theta}(x) \in \mathbb{R}^{|\mathcal{V}|}$, and a label $y \in \mathcal{V}$, the log-probability of y is

$$\log p_{\theta}(y \mid x) = \log [\text{softmax}(f_{\theta}(x))]_y, \quad (3)$$

where the notation $[x]_y$ denotes the y -th element of the vector x .

You will want to use a numerically stable method to compute this, and are free to use methods from `torch.nn.functional`. We also suggest including an argument to optionally compute and return token entropies.

Problem (`get_response_log_probs`): Response log-probs (and entropy)

Deliverable: Implement a method `get_response_log_probs` that gets per-token conditional log-probabilities (given the previous tokens) from a causal language model, and optionally the entropy of the model's next-token distribution. The following interface is recommended:

```
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: torch.Tensor,
    labels: torch.Tensor,
    return_token_entropy: bool = False,
) -> dict[str, torch.Tensor]:
    """
    Args:

        model: PreTrainedModel HuggingFace model used for scoring (placed on
            the correct device
            and in inference mode if gradients should not be computed).

        input_ids: torch.Tensor shape (batch_size, sequence_length),
            concatenated prompt +
            response tokens as produced by your tokenization method.

        labels: torch.Tensor shape (batch_size, sequence_length), labels as
            produced by your
            tokenization method.

        return_token_entropy: bool If True, also return per-token entropy by
            calling
            compute_entropy.

    Returns:

        dict[str, torch.Tensor].

        "log_probs" shape (batch_size, sequence_length), conditional log-
            probabilities
             $-\log p_{\theta}(x_t \mid x_{<t})$ .

        "token_entropy" optional, shape (batch_size, sequence_length),
            per-token entropy
            for each position (present only if return_token_entropy=True).

    """
```

Implementation tips:

- Obtain logits with `model(input_ids).logits`.

To test your code, implement `[adapters.run_get_response_log_probs]`. Then run `uv run pytest -k test_get_response_log_probs` and ensure the test passes.

We implement a helper method to ensure only non-masked tokens contribute to the loss.

Problem (masked_normalize): Masked normalize

Deliverable: Implement a method `masked_normalize` that sums over tensor elements and normalizes by a constant while respecting a boolean mask. The following interface is recommended:

```
def masked_normalize(
    tensor: torch.Tensor,
    mask: torch.Tensor,
    normalize_constant: float,
    dim: int | None = None,
) -> torch.Tensor:
    """
    Sum over a dimension and normalize by a constant, considering only those
    elements where mask
    == 1.

    Args:

        tensor: torch.Tensor The tensor to sum and normalize.

        mask: torch.Tensor Same shape as tensor; positions with 1 are
            included in the sum.

        normalize_constant: float the constant to divide by for normalization
            .

        dim: int | None the dimension to sum along before normalization. If
            None, sum over all
            dimensions.

    Returns:

        torch.Tensor the normalized sum, where masked elements (mask == 0)
            don't contribute to
            the sum.
    """
```

To test your code, implement `[adapters.run_masked_normalize]`. Then run `uv run pytest -k test_masked_normalize` and ensure it passes.

Logging generations in-the-loop. It's always good practice to do some in-the-loop logging that involves generation from your model, and reasoning RL is no exception. Write a function `log_generations` that will prompt your model to generate responses for some given prompts (e.g., sampled from the validation set). It's a good idea to log at least the following for each example:

1. The input prompt.
2. The response generated by the RL model.
3. The ground-truth answer.
4. The reward information, including format, answer, and total reward.
5. The average token entropy of the response.
6. The average response length, average response length for correct responses, and average response length for incorrect responses.

Problem (log_generations): Logging generations

Deliverable: Implement a function `log_generations` that can be used to log generations from your model.

3.4 Policy Gradients Recap

In this section, we provide additional information on the theory behind training LLMs with reinforcement learning. There are no deliverables in this section, it simply includes content that may be useful for the later sections of the assignment.

3.4.1 Language Models as Policies

A causal language model (LM) with parameters θ defines a probability distribution over the next token $a_t \in \mathcal{V}$ given the current text prefix s_t (the state/observation). In the context of RL, we think of the next token a_t as an action and the current text prefix s_t as the state. Hence, the LM is a *categorical stochastic policy*

$$a_t \sim \pi_\theta(\cdot \mid s_t), \quad \pi_\theta(a_t \mid s_t) = [\text{softmax}(f_\theta(s_t))]_{a_t}. \quad (4)$$

Two primitive operations will be needed in optimizing the policy with policy gradients:

1. *Sampling from the policy:* drawing an action a_t from the categorical distribution above;
2. *Scoring the log-likelihood of an action:* evaluating $\log \pi_\theta(a_t \mid s_t)$.

Generally, when doing RL with LLMs, s_t is the partial completion/solution produced so far, and each a_t is the next token of the solution; the episode ends when an end-of-text token is emitted, like `<|end_of_text|>`, or `</answer>` in the case of our prompt.

3.4.2 Trajectories

A (finite-horizon) trajectory is the interleaved sequence of states and actions experienced by an agent:

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T), \quad (5)$$

where T is the length of the trajectory, i.e., a_T is an end-of-text token or we have reached a maximum generation budget in tokens.

The initial state is drawn from the start distribution, $s_0 \sim \rho_0(s_0)$; in the case of RL with LLMs, $\rho_0(s_0)$ is a distribution over formatted prompts. dynamics $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. In RL with

LLMs, the environment is deterministic: the next state is the old prefix concatenated with the emitted token, $s_{t+1} = s_t \| a_t$. Trajectories are also called *episodes* or *rollouts*; we will use these terms interchangeably.

3.4.3 Rewards and Return

A scalar reward $r_t = R(s_t, a_t)$ judges the immediate quality of the action taken at state s_t . For RL on verified domains, it is standard to assign zero reward to intermediate steps and a *verified reward* to the terminal action

$$r_T = R(s_T, a_T) := \begin{cases} 1 & \text{if the trajectory } s_T \| a_T \text{ matches the ground-truth} \\ 0 & \text{otherwise.} \end{cases}$$

The *return* $R(\tau)$ aggregates rewards along the trajectory. Two common choices are *finite-horizon undiscounted returns*

$$R(\tau) := \sum_{t=0}^T r_t, \quad (6)$$

and *infinite-horizon discounted returns*

$$R(\tau) := \sum_{t=0}^{\infty} \gamma^t r_t, \quad 0 < \gamma < 1. \quad (7)$$

In our case, we will use the undiscounted formulation since episodes have a natural termination point (end-of-text or max generation length).

The objective of the agent is to maximize the expected return

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)], \quad (8)$$

leading to the optimization problem

$$\theta^* = \arg \max_{\theta} J(\theta). \quad (9)$$

3.4.4 Vanilla Policy Gradient

Next, let us attempt to learn policy parameters θ with *gradient ascent* on the expected return:

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\theta_k). \quad (10)$$

The core identity that we will use to do this is the REINFORCE policy gradient, shown below.

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]. \quad (11)$$

Deriving the policy gradient. How did we get this equation? For completeness, we will give a derivation of this identity below. We will make use of a few identities.

1. The probability of a trajectory is given by

$$P(\tau | \theta) = \rho_0(s_0) \prod_{t=0}^T P(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t). \quad (12)$$

Therefore, the log-probability of a trajectory is:

$$\log P(\tau \mid \theta) = \log \rho_0(s_0) + \sum_{t=0}^T [\log P(s_{t+1} \mid s_t, a_t) + \log \pi_\theta(a_t \mid s_t)]. \quad (13)$$

2. The log-derivative trick:

$$\nabla_\theta P = P \nabla_\theta \log P. \quad (14)$$

3. The environment terms are constant in θ . ρ_0 , $P(\cdot \mid \cdot)$ and $R(\tau)$ do not depend on the policy parameters, so

$$\nabla_\theta \rho_0 = \nabla_\theta P = \nabla_\theta R(\tau) = 0. \quad (15)$$

Applying the facts above:

$$\nabla_\theta J(\theta) = \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \quad (16)$$

$$= \nabla_\theta \sum_{\tau} P(\tau \mid \theta) R(\tau) \quad (17)$$

$$= \sum_{\tau} \nabla_\theta P(\tau \mid \theta) R(\tau) \quad (18)$$

$$= \sum_{\tau} P(\tau \mid \theta) \nabla_\theta \log P(\tau \mid \theta) R(\tau) \quad (\text{Log-derivative trick}) \quad (19)$$

$$= \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log P(\tau \mid \theta) R(\tau)], \quad (20)$$

and therefore, plugging in the log-probability of a trajectory and using the fact that the environment terms are constant in θ , we get the vanilla REINFORCE policy gradient:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t \mid s_t) R(\tau) \right]. \quad (21)$$

Intuitively, this gradient will increase the log probability of every action in a trajectory that has high return, and decrease them otherwise.

Sample estimate of the gradient. Given a batch of N rollouts $\mathcal{D} = \{\tau^{(i)}\}_{i=1}^N$ collected by sampling a starting state $s_0^{(i)} \sim \rho_0(s_0)$ and then running the policy π_θ in the environment, we form an unbiased estimator of the gradient as

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(i)} \mid s_t^{(i)}) R(\tau^{(i)}). \quad (22)$$

This vector is used in the gradient-ascent update $\theta \leftarrow \theta + \alpha \hat{g}$.

3.4.5 Policy Gradient Baselines

The main issue with vanilla policy gradient is the high variance of the gradient estimate. A common technique to mitigate this is to subtract from the reward a *baseline* function b that depends only on the state. This is a type of *control variate* [5]: the idea is to decrease the variance of the estimator by subtracting a term that is correlated with it, without introducing bias.

Let us define the baselined policy gradient as:

$$B = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) (R(\tau) - b(s_t)) \right]. \quad (23)$$

As an example, a reasonable baseline is the on-policy value function $V^\pi(s) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau) | s_t = s]$, i.e., the expected return if we start at $s_t = s$ and follow the policy π_θ from there. Then, the quantity $(R(\tau) - V^\pi(s_t))$ is, intuitively, how much better the realized trajectory is than expected.

As long as the baseline depends only on the state, the baselined policy gradient is unbiased. We can see this by rewriting the baselined policy gradient as

$$B = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau) \right] - \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t) \right]. \quad (24)$$

Focusing on the baseline term, we see that

$$\mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) b(s_t) \right] = \sum_{t=0}^T \mathbb{E}_{s_t} [b(s_t) \mathbb{E}_{a_t \sim \pi_\theta(\cdot | s_t)} [\nabla_\theta \log \pi_\theta(a_t | s_t)]] . \quad (25)$$

In general, the expectation of the score function is zero: $\mathbb{E}_{x \sim P_\theta} [\nabla_\theta \log P_\theta(x)] = 0$. Therefore, the expression in Eq. 24 is zero and

$$B = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) R(\tau) \right] - 0 = \nabla_\theta J(\pi_\theta), \quad (26)$$

so we conclude that the baselined policy gradient is unbiased. We will later run an experiment to see whether baselining improves downstream performance.

A note on policy gradient “losses”. When we implement policy gradient methods in a framework like PyTorch, we will define a so-called policy gradient loss `pg_loss` such that calling `pg_loss.backward()` will populate the gradient buffers of our model parameters with our approximated policy gradient $\hat{g}$. In math, it can be stated as

$$\text{pg_loss} = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) (R(\tau^{(i)}) - b(s_t^{(i)})). \quad (27)$$

`pg_loss` is not a loss in the canonical sense—it’s not meaningful to report `pg_loss` on the train or validation set as an evaluation metric, and a good validation `pg_loss` doesn’t indicate that our model is generalizing well. The `pg_loss` is really just some scalar such that when we call `pg_loss.backward()`, the gradients we obtain through backprop are the approximate policy gradient $\hat{g}$.

When doing RL, you should always log and report train and validation rewards. These are the “meaningful” evaluation metrics and what we are attempting to optimize with policy gradient methods.

3.5 Group Relative Policy Optimization

Next, we will describe Group Relative Policy Optimization (GRPO), the variant of policy gradient that you will implement and experiment with for solving math problems.

Algorithm 1 Group Relative Policy Optimization (GRPO)

```
1: Input initial policy model  $\pi_{\theta_{\text{init}}}$ , reward function  $R$ ; task questions  $\mathcal{D}$ 
2: policy model  $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$ 
3: for step = 1, ..., n_grpo_steps do
4:   Sample a batch of questions  $\mathcal{D}_b$  from  $\mathcal{D}$ 
5:   Set the old policy model  $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ 
6:   Sample  $G$  outputs  $\{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$  for each question  $q \in \mathcal{D}_b$ 
7:   Compute rewards  $\{r^{(i)}\}_{i=1}^G$  for each sampled output  $o^{(i)}$  by running reward function  $R(q, o^{(i)})$ 
8:   Compute  $A^{(i)}$  with group normalization (Eq. 28)
9:   for train step = 1, ..., n_train_steps_per_rollout_batch do
10:    Update the policy model  $\pi_{\theta}$  by maximizing the GRPO-Clip objective (to be discussed,
    Eq. 29)
11:   end for
12: end for
13: Output  $\pi_{\theta}$ 
```

3.5.1 GRPO Algorithm

Advantage estimation. The core idea of GRPO is to sample many outputs for each question from the policy π_{θ} and use them to compute a baseline. This is convenient because we avoid the need to learn a neural value function $V_{\phi}(s)$, which can be hard to train and is cumbersome from the systems perspective. For a question q and group outputs $\{o^{(i)}\}_{i=1}^G \sim \pi_{\theta}(\cdot | q)$, let $r^{(i)} = R(q, o^{(i)})$ be the reward for the i -th output. DeepSeekMath [7] and DeepSeek R1 [6] compute the group-normalized reward for the i -th output as

$$A^{(i)} = \frac{r^{(i)} - \text{mean}(r^{(1)}, r^{(2)}, \dots, r^{(G)})}{\text{std}(r^{(1)}, r^{(2)}, \dots, r^{(G)}) + \text{advantage_eps}}, \quad (28)$$

where `advantage_eps` is a small constant to prevent division by zero. Note that this *advantage* $A^{(i)}$ is the same for each token in the response, i.e., $A_t^{(i)} = A^{(i)}$, $\forall t \in 1, \dots, |o^{(i)}|$, so we drop the t subscript in the following.

High-level algorithm. Before we dive into the GRPO objective, let us first get an idea of the train loop by writing out the algorithm from Shao et al. [7] in Algorithm 1.¹

GRPO objective. The GRPO objective combines three ideas:

1. An importance-ratio / PPO-style clipped surrogate objective that compares the current policy to the rollout-generating policy.
2. Computing advantages $A^{(i)}$ with group normalization, as in Eq. 28.
3. A clipping mechanism, as in Proximal Policy Optimization (PPO, Schulman et al. [8]).

The purpose of clipping is to maintain stability when taking many gradient steps on a single batch of rollouts. It works by keeping the policy π_{θ} from straying too far from the old policy.

¹This is a special case of DeepSeekMath’s GRPO with a verified reward function, no KL term, and no iterative update of the reference and reward model.

Let us first write out the full GRPO-Clip objective, and then we can build some intuition on what the clipping does:

$$J_{\text{GRPO-Clip}}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, \{o^{(i)}\}_{i=1}^G \sim \pi_\theta(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o^{(i)}|} \sum_{t=1}^{|o^{(i)}|} \min \left(\frac{\pi_\theta(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})} A^{(i)}, \text{clip} \left(\frac{\pi_\theta(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})}, 1 - \epsilon, 1 + \epsilon \right) A^{(i)} \right) \right] \quad (29)$$

The hyperparameter $\epsilon > 0$ controls how much the policy can change. To see this, we can rewrite the per-token objective in a more intuitive way following Achiam [9, 10]. Define the function

$$g(\epsilon, A^{(i)}) = \begin{cases} (1 + \epsilon)A^{(i)} & \text{if } A^{(i)} \geq 0 \\ (1 - \epsilon)A^{(i)} & \text{if } A^{(i)} < 0. \end{cases} \quad (30)$$

We can rewrite the per-token objective as

$$\text{per-token objective} = \min \left(\frac{\pi_\theta(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})} A^{(i)}, g(\epsilon, A^{(i)}) \right).$$

We can now reason by cases. When the advantage $A^{(i)}$ is positive, the per-token objective simplifies to

$$\text{per-token objective} = \min \left(\frac{\pi_\theta(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})}, 1 + \epsilon \right) A^{(i)}.$$

Since $A^{(i)} > 0$, the objective goes up if the action $o_t^{(i)}$ becomes more likely under π_θ , i.e., if $\pi_\theta(o_t^{(i)} | q, o_{<t}^{(i)})$ increases. The clipping with $\min$ limits how much the objective can increase: once $\pi_\theta(o_t^{(i)} | q, o_{<t}^{(i)}) > (1 + \epsilon)\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})$, this per-token objective hits its maximum value of $(1 + \epsilon)A^{(i)}$. So, the policy π_θ is not incentivized to go very far from the old policy $\pi_{\theta_{\text{old}}}$.

Analogously, when the advantage $A^{(i)}$ is negative, the model tries to drive down $\pi_\theta(o_t^{(i)} | q, o_{<t}^{(i)})$, but is not incentivized to decrease it below $(1 - \epsilon)\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})$ (refer to Achiam [10] for the full argument).

3.5.2 Implementation

Now that we have a high-level understanding of the GRPO training loop and objective, we will start implementing pieces of it.

Computing advantages (group-normalized rewards). First, we will implement the logic to compute advantages for each example in a rollout batch, i.e., the group-normalized rewards. We will consider two possible ways to obtain group-normalized rewards: the approach presented above in Eq. 28, and a recent simplification of this idea.

Dr. GRPO [12] highlights that normalizing by $\text{std}(r^{(1)}, r^{(2)}, \dots, r^{(G)})$ rewards questions in a batch with low variation in answer correctness, but this may not be desirable. They propose simply removing the normalization step, computing

$$A^{(i)} = r^{(i)} - \text{mean}(r^{(1)}, r^{(2)}, \dots, r^{(G)}). \quad (31)$$

We will implement both variants and compare their performance later in the assignment.

Problem (compute_group_normalized_rewards): Group normalization

Deliverable: Implement a method `compute_group_normalized_rewards` that calculates raw rewards for each rollout response, normalizes them into their groups, and returns both the normalized and raw rewards along with any metadata you think is useful. The following interface is recommended:

```
def compute_group_normalized_rewards(
    reward_fn,
    rollout_responses,
    repeated_ground_truths,
    group_size,
    advantage_eps,
    normalize_by_std,
):
```

Compute rewards for each group of rollout responses, normalized by the group size.

Args:

- `reward_fn`: Callable[[str, str], dict[str, float]] Scores the rollout responses against the ground truths, producing a dict with keys "reward", "format_reward", and "answer_reward".
- `rollout_responses`: list[str] Rollouts from the policy. The length of this list is `rollout_batch_size = n_prompts_per_rollout_batch * group_size`.
- `repeated_ground_truths`: list[str] The ground truths for the examples. The length of this list is `rollout_batch_size`, because the ground truth for each example is repeated `group_size` times.
- `group_size`: int Number of responses per question (group).
- `advantage_eps`: float Small constant to avoid division by zero in normalization.
- `normalize_by_std`: bool If True, divide by the per-group standard deviation; otherwise subtract only the group mean.

Returns:

- tuple[torch.Tensor, torch.Tensor, dict[str, float]]
- `advantages` shape (rollout_batch_size,). Group-normalized rewards for each rollout response.
- `raw_rewards` shape (rollout_batch_size,). Unnormalized rewards for each rollout response.
- `metadata` your choice of other statistics to log (e.g. mean, std, max/min of rewards).

To test your code, implement `[adapters.run_compute_group_normalized_rewards]`. Then run the test with `uv run pytest -k test_compute_group_normalized_rewards` and make sure your implementation passes it.

GRPO-Clip loss. Next, we will implement the more interesting GRPO-Clip loss.

The per-token GRPO-Clip loss is

$$-\min\left(\frac{\pi_{\theta}(o_t | q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t | q, o_{<t})} A_t, \text{clip}\left(\frac{\pi_{\theta}(o_t | q, o_{<t})}{\pi_{\theta_{\text{old}}}(o_t | q, o_{<t})}, 1 - \epsilon, 1 + \epsilon\right) A_t\right). \quad (32)$$

Problem (compute_grpo_clip_loss): GRPO-Clip loss

Deliverable: Implement a method `compute_grpo_clip_loss` that computes the per-token GRPO-Clip loss. The following interface is recommended:

```
def compute_grpo_clip_loss(
    advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
    old_log_probs: torch.Tensor,
    cliprange: float,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

Args:

- `advantages`: `torch.Tensor` Shape `(batch_size, 1)`, per-example advantages A .
- `policy_log_probs`: `torch.Tensor` Shape `(batch_size, sequence_length)`, per-token log probs from the policy being trained.
- `old_log_probs`: `torch.Tensor` Shape `(batch_size, sequence_length)`, per-token log probs from the old policy.
- `cliprange`: `float` Clip parameter ϵ (e.g. 0.2).

Returns:

- `tuple[torch.Tensor, dict[str, torch.Tensor]]`.
- `loss torch.Tensor` of shape `(batch_size, sequence_length)`, the per-token clipped loss.
- `metadata` dict containing whatever you want to log. We suggest logging whether each token was clipped or not, i.e., whether the clipped policy gradient loss on the RHS of the min was lower than the LHS.

Implementation tips:

- Broadcast advantages over `sequence_length`.

To test your code, implement `[adapters.run_compute_grpo_clip_loss]`. Then run `uv run pytest -k test_compute_grpo_clip_loss` and ensure the test passes.

GRPO microbatch train step. Now we are ready to implement a single microbatch train step for GRPO (recall that for a train minibatch, we iterate over many microbatches if `gradient_accumulation_steps > 1`).

Specifically, given the advantages and log probs, we will compute the per-token GRPO-Clip loss, use `masked_mean` to aggregate to a scalar loss per example, average over the batch dimension, adjust for gradient accumulation, and backpropagate.

Problem (grpo_microbatch_train_step): Microbatch train step

Deliverable: Implement a single micro-batch update for GRPO, including GRPO-Clip loss, averaging with a mask, and gradient scaling.

The following interface is recommended:

```
def grpo_microbatch_train_step(
    policy_log_probs: torch.Tensor,
    response_mask: torch.Tensor,
    gradient_accumulation_steps: int,
    advantages: torch.Tensor,
    old_log_probs: torch.Tensor,
    cliprange: float,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

Execute a forward-and-backward pass on a microbatch.

Args:

- `policy_log_probs` (`batch_size`, `sequence_length`), per-token log-probabilities from the policy being trained.
- `response_mask` (`batch_size`, `sequence_length`), 1 for response tokens, 0 for prompt/padding.
- `gradient_accumulation_steps` Number of microbatches per optimizer step.
- `advantages` (`batch_size`, 1), per-example advantages.
- `old_log_probs` (`batch_size`, `sequence_length`), per-token log-probabilities from the old policy.
- `cliprange` Clip parameter ϵ for GRPO-Clip.

Returns:

- `tuple[torch.Tensor, dict[str, torch.Tensor]]`.
- `loss` scalar tensor. The microbatch loss, adjusted for gradient accumulation. We return this so we can log it.
- `metadata` Dict with metadata from the underlying loss call, and any other statistics you might want to log.

Implementation tips:

- You should call `loss.backward()` in this function.
- Make sure to adjust for gradient accumulation.

To test your code, implement `[adapters.run_grpo_microbatch_train_step]`. Then run `uv run pytest -k test_grpo_microbatch_train_step` and confirm it passes.

Putting it all together: GRPO train loop. Now we will put together a complete train loop for GRPO. You should refer to the algorithm in Section 7.1 for the overall structure, using the methods we've implemented where appropriate.

Below we provide some starter hyperparameters. If you have a correct implementation, you should see reasonable results with these. For reference, the staff implementation takes around 1 hour for 50 iterations and achieves an average validation reward of around 0.68.

```
n_grpo_steps: int = 8
learning_rate: float = 1e-5

advantage_eps: float = 1e-6
rollout_batch_size: int = 32
group_size: int = 8
sampling_temperature: float = 1.0
sampling_min_tokens: int = 4 # As in Eptert, disallow empty string responses
sampling_max_tokens: int = 256
epochs_per_rollout_batch: int = 1 # On-policy
train_batch_size: int = 32 # On-policy
gradient_accumulation_steps: int = 16
cliprange: float = 1.0

optimizer = torch.optim.Adam(
    policy.parameters(),
    lr=learning_rate,
    weight_decay=0.0,
    betas=(0.9, 0.95),
)

assert train_batch_size % gradient_accumulation_steps == 0, (
    "train_batch_size must be divisible by gradient_accumulation_steps"
)

micro_train_batch_size = train_batch_size // gradient_accumulation_steps
assert rollout_batch_size % group_size == 0, (
    "rollout_batch_size must be divisible by group_size"
)

n_prompts_per_rollout_batch = rollout_batch_size // group_size
assert train_batch_size >= group_size, (
    "train_batch_size must be greater than or equal to group_size"
)

n_microbatches_per_rollout_batch = rollout_batch_size // micro_train_batch_size
```

These default hyperparameters will start you in the on-policy setting—for each rollout batch, we take a single gradient step. In terms of hyperparameters, this means that `train_batch_size` is equal to `rollout_batch_size`, and `epochs_per_rollout_batch` is equal to 1.

Here are some sanity check asserts and constants that should remove some edge cases and point you in the right direction:

- We recommend using the CoT prompt, and to stop generation at the second answer tag `</answer>`, as in the previous experiments.
- We suggest using `return_token_entropy=False`.
- Use gradient clipping with clip value 1.0.

- You should routinely log validation rewards (e.g. every 5 or 10 steps). You should evaluate on at least 256 validation examples to compare hyperparameters, as CoT/RL evaluations can be noisy.
- GRPO-Clip requires `old_log_probs`, i.e. the log-probabilities under the policy that generated the rollout batch. This is fully compatible with the standard on-policy setting: sample rollouts from the current policy, cache their log-probabilities as `old_log_probs`, then optimize against those fixed values.
- In the off-policy setting with multiple epochs of gradient updates per rollout batch, it would be wasteful to recompute the old log-probabilities for each epoch. Instead, we can compute the old log-probabilities once and reuse them for each epoch.
- You should not differentiate with respect to the old log-probabilities.
- You should log some or all of the following for each optimizer update:
 - The loss.
 - Gradient norm.
 - Token entropy.
 - Clip fraction, if off-policy.
 - Train rewards (total, format, and answer).
 - Anything else you think could be useful for debugging.

Problem (`grpo_train_loop`): GRPO train loop

Deliverable: Implement a complete train loop for GRPO using only the GRPO-Clip objective. Begin training a policy on the training set of GSM8K and confirm that you see validation rewards improving, along with sensible rollouts over time. Provide a plot with the validation rewards with respect to steps, and a few example rollouts over time. Due to compute limitations, please train your model for 50 iterations.

Normalization with group standard deviation. Recall our standard implementation of `compute_group_normalized_rewards` (based on Shao et al. [7], DeepSeek-AI et al. [6]), where we normalized by the group standard deviation. Liu et al. [11] note that dividing by the group standard deviation could introduce unwanted biases to the training procedure: questions with lower standard deviations (e.g., too easy or too hard questions with all rewards almost all 1 or all 0) would receive higher weights during training.

Liu et al. [11] propose removing the normalization by the standard deviation, which we have already implemented in `compute_group_normalized_rewards` and will now test.

Problem (`grpo_group_standard_deviation`): Effect of standard deviation normalization

Deliverable: Compare the performance of `use_std_normalization == True` and `use_std_normalization == False`. Report the validation answer reward curves. Comment on the findings, including any other metrics that have a noticeable trend. Due to compute limitations, please train your model for 50 iterations, which should be sufficient to notice differences.

Hint: consider metrics related to stability, such as the gradient norm.

References

- [1] Cobbe, Karl, et al. "Training verifiers to solve math word problems." arXiv preprint arXiv:2110.14168 (2021).
- [2] Wei, Jason, et al. "Chain-of-thought prompting elicits reasoning in large language models." *Advances in neural information processing systems* 35 (2022): 24824-24837.
- [3] Wang, Xuezhi, et al. "Self-consistency improves chain of thought reasoning in language models." arXiv preprint arXiv:2203.11171 (2022).
- [4] Shao, Zhihong, et al. "Deepseekmath: Pushing the limits of mathematical reasoning in open language models." arXiv preprint arXiv:2402.03300 (2024).
- [5] Ross, Sheldon M. *Simulation*. Academic Press, 2022.
- [6] DeepSeek-AI. "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning." arXiv preprint arXiv:2501.12948, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [7] Shao, Zhihong. "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models." arXiv preprint arXiv:2402.03300, 2024. URL <https://arxiv.org/abs/2402.03300>.
- [8] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. "Proximal policy optimization algorithms." arXiv preprint arXiv:1707.06347, 2017. URL <https://arxiv.org/abs/1707.06347>.
- [9] Achiam, Joshua. *Spinning Up in Deep Reinforcement Learning*. 2018.
- [10] Achiam, Joshua. "Simplified PPO-Clip Objective." 2018. URL <https://drive.google.com/file/d/1PDzn9RPvaXjJFZkGeapMHbHGiWW20Ey/view>.
- [11] Liu, Zichen. "Understanding R1-Zero-Like Training: A Critical Perspective." arXiv preprint arXiv:2503.20783, 2025. URL <https://arxiv.org/abs/2503.20783>.
- [12] Zeng, Weihao. "SimpleRL-Zoo: Investigating and taming zero reinforcement learning for open base models in the wild." arXiv preprint arXiv:2503.18892, 2025. URL <https://arxiv.org/abs/2503.18892>.